

Shared Python csomag - Osztály szintű dokumentáció

April 30, 2026



- shared package
 - APIKey
 - * APIKey.Relationships
 - * APIKey.project
 - * APIKey.quotas
 - * APIKey.user
 - Limit

- * Limit.Relationships
 - * Limit.quotas
- Period
 - * Period.DAY
 - * Period.HOUR
 - * Period.MINUTE
 - * Period.MONTH
 - * Period.WEEK
- Project
 - * Project.Relationships
 - * Project.api_keys
 - * Project.permissions
 - * Project.quotas
- ProjectPermission
 - * ProjectPermission.Relationships
 - * ProjectPermission.project
 - * ProjectPermission.role
 - * ProjectPermission.user
- Quota
 - * Quota.Relationships
 - * Quota.api_key
 - * Quota.limit
 - * Quota.limit_value
 - * Quota.project
 - * Quota.user
- Role
 - * Role.Relationships
 - * Role.permissions
- SQLAlchemyBase
- Status
 - * Status.ACTIVE
 - * Status.ARCHIVED
 - * Status.BLOCKED
 - * Status.EXPIRED
- UsageLog
 - * UsageLog.Relationships
 - * UsageLog.api_key
 - * UsageLog.project
 - * UsageLog.user
- User
 - * User.Relationships
 - * User.api_keys

- * User.permissions
 - * User.quotas
- hash_key()
- verify_key()
- Subpackages
 - * shared.config package
 - Submodules
 - shared.config.config module
 - * shared.db package
 - get_session()
 - get_transactional_session()
 - Submodules
 - shared.db.db module
 - * shared.models package
 - APIKey
 - Limit
 - Period
 - Project
 - ProjectPermission
 - Quota
 - Role
 - SQLAlchemyBase
 - Status
 - UsageLog
 - User
 - Subpackages
 - * shared.utils package
 - calculate_date_after_period()
 - find_project_root()
 - hash_key()
 - hash_password()
 - verify_key()
 - verify_password()
 - Submodules
 - shared.utils.date_calc module
 - shared.utils.hashing module
 - shared.utils.password module
 - shared.utils.pathing module

2 shared.config package

2.1 Submodules

2.2 shared.config.config module

3 shared.db package

3.0.1 `get_session()` $\rightarrow$ `AsyncGenerator[sqlalchemy.ext.asyncio.AsyncSession, None]`

Provides an asynchronous context manager for obtaining a SQLAlchemy session. It creates a new session using the SessionLocal factory and yields it for use within the context. The session is automatically closed when the context is exited.

- **Returns:** An asynchronous SQLAlchemy session for interacting with the database.
- **Return type:** AsyncSession

3.0.2 `get_transactional_session()` $\rightarrow$ `AsyncGenerator[sqlalchemy.ext.asyncio.AsyncSession, None]`

Provides an asynchronous context manager for obtaining a transactional SQLAlchemy session. It creates a new session using the SessionLocal factory and yields it for use within the context. The session is automatically closed when the context is exited.

- **Returns:** An asynchronous transactional SQLAlchemy session for interacting with the database.
- **Return type:** AsyncSession

3.1 Submodules

3.2 shared.db.db module

3.2.1 `get_session()` $\rightarrow$ `AsyncGenerator[sqlalchemy.ext.asyncio.AsyncSession, None]`

Provides an asynchronous context manager for obtaining a SQLAlchemy session. It creates a new session using the SessionLocal factory and yields it for use within the context. The session is automatically closed when the context is exited.

- **Returns:** An asynchronous SQLAlchemy session for interacting with the database.
- **Return type:** AsyncSession

3.2.2 `get_transactional_session()` → `AsyncGenerator[sqlalchemy.ext.asyncio.AsyncSession, None]`

Provides an asynchronous context manager for obtaining a transactional SQLAlchemy session. It creates a new session using the `SessionLocal` factory and yields it for use within the context. The session is automatically closed when the context is exited.

- **Returns:** An asynchronous transactional SQLAlchemy session for interacting with the database.
- **Return type:** `AsyncSession`

4 shared package

4.0.1 `class APIKey(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing an API key in the database.

4.0.2 - `id`

- **Type:** The unique identifier for the API key (primary key).

4.0.3 - `project_id`

- **Type:** The foreign key linking to the associated project.

4.0.4 - `user_id`

- **Type:** The foreign key linking to the associated user, nullable for keys not tied to a specific user.

4.0.5 - `name`

- **Type:** A human-readable name for the API key.

4.0.6 - `fingerprint`

- **Type:** A unique fingerprint derived from the API key for quick lookup.

4.0.7 - `key_hash`

- **Type:** A secure hash of the API key for verification purposes.

4.0.8 - `create_date`

- **Type:** The timestamp when the API key was created.

4.0.9 - status

- **Type:** The current status of the API key (e.g., active, inactive).

Relationships

4.0.10 ———-

4.0.11 - project

- **Type:** The relationship to the Project model, indicating which project this API key belongs to.

4.0.12 - user

- **Type:** The relationship to the User model, indicating which user this API key belongs to (if applicable).

4.0.13 - quotas

- **Type:** The relationship to the Quota model, representing the quotas directly targeting the API key.

`project = 'Project'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

`user = 'User'`

4.0.14 *class* Limit(*args: Any, **kwargs: Any)

Bases: `Base`

ORM model representing a quota limit type's definition.

4.0.15 - id

- **Type:** The unique identifier for the limit (primary key).

4.0.16 - name

- **Type:** A unique name for the limit type (for example 'Token Limit' or 'Request Limit').

4.0.17 - description

- **Type:** A human-readable description of what the limit controls.

Relationships

4.0.18 ———-

4.0.19 - quotas

- **Type:** The relationship to Quota rows that reference this limit type.

quotas : *sqlalchemy.orm.Mapped.list['Quota']* = *'Quota'*

4.0.20 *class* Period(*values)

Bases: Enum

Enumeration of supported quota reset periods.

4.1 Values

- MINUTE
- HOUR
- DAY
- WEEK
- MONTH

DAY = *'day'*

HOUR = *'hour'*

MINUTE = *'minute'*

MONTH = *'month'*

WEEK = *'week'*

4.1.1 *class* Project(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a project in the database.

4.1.2 - id

- **Type:** The unique identifier for the project (primary key).

4.1.3 - name

- **Type:** The unique, human-readable project name.

4.1.4 - status

- **Type:** The lifecycle status of the project (e.g., ACTIVE, ARCHIVED).

4.1.5 - created_date

- **Type:** The timestamp when the project was created.

4.1.6 - modified_date

- **Type:** The timestamp of the latest update, if any.

Relationships

4.1.7 ————

4.1.8 - permissions

- **Type:** The relationship to ProjectPermission rows linked to this project.

4.1.9 - api_keys

- **Type:** The relationship to APIKey rows issued under this project.

4.1.10 - quotas

- **Type:** The relationship to Quota rows directly targeting this project.

`api_keys : sqlalchemy.orm.Mapped.list['APIKey'] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

4.1.11 `class ProjectPermission(*args: Any, **kwargs: Any)`

Bases: Base

ORM model representing a user's role assignment within a project.

4.1.12 - project_id

- **Type:** The foreign key to the associated project (composite primary key).

4.1.13 - user_id

- **Type:** The foreign key to the associated user (composite primary key).

4.1.14 - role_id

- **Type:** The foreign key to the role granted in the project.

4.1.15 - join_date

- **Type:** The timestamp when the permission assignment was created.

Relationships

4.1.16 ———-

4.1.17 - project

- **Type:** The relationship to the Project receiving the permission mapping.

4.1.18 - user

- **Type:** The relationship to the User receiving the role in the project.

4.1.19 - role

- **Type:** The relationship to the Role granted for the project membership.

`project = 'Project'`

`role = 'Role'`

`user = 'User'`

4.1.20 *class* Quota(*args: Any, **kwargs: Any)

Bases: `Base`

ORM model representing quota allocation and reset state.

4.1.21 - id

- **Type:** The unique identifier for the quota entry (primary key).

4.1.22 - project_id

- **Type:** Optional foreign key to the project targeted by this quota.

4.1.23 - user_id

- **Type:** Optional foreign key to the user targeted by this quota.

4.1.24 - key_id

- **Type:** Optional foreign key to the API key targeted by this quota.

4.1.25 - limit_id

- **Type:** The foreign key to the limit type being constrained.

4.1.26 - limit_value

- **Type:** The allocated maximum value for the configured limit and period.

4.1.27 - period

- **Type:** The reset period for the quota.

4.1.28 - expires_at

- **Type:** Optional timestamp when this quota definition expires.

4.1.29 - status

- **Type:** The current lifecycle state of the quota.

4.1.30 - allocated

- **Type:** The currently consumed amount within the active period.

4.1.31 - next_reset

- **Type:** The timestamp when allocated usage resets.

Relationships

4.1.32 ———-

4.1.33 - project

- **Type:** The relationship to the Project this quota directly applies to.

4.1.34 - api_key

- **Type:** The relationship to the APIKey this quota directly applies to.

4.1.35 - user

- **Type:** The relationship to the User this quota directly applies to.

4.1.36 - limit

- **Type:** The relationship to the Limit definition for this quota.

api_key = 'APIKey'

`limit = 'Limit'`

`limit_value : sqlalchemy.orm.Mapped[float] / None`

`project = 'Project'`

`user = 'User'`

4.1.37 `class Role(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a project membership role's definition.

4.1.38 - `id`

- **Type:** The unique identifier for the role (primary key).

4.1.39 - `name`

- **Type:** A unique role name.

4.1.40 - `description`

- **Type:** A human-readable description of role permissions.

Relationships

4.1.41 `_____`

4.1.42 - `permissions`

- **Type:** The relationship to `ProjectPermission` rows using this role.

`permissions : sqlalchemy.orm.Mapped[list['ProjectPermission']] = 'ProjectPermission'`

4.1.43 `SQLAlchemyBase`

alias of `Base`

4.1.44 `class Status(*values)`

Bases: `Enum`

Enumeration of lifecycle states for ORM entities.

4.2 Values

- **ACTIVE:** The entity is active and operational.
- **BLOCKED:** The entity is blocked and should not be used. (e.g., due to policy violations or security concerns)
- **ARCHIVED:** The entity is archived and no longer active. It may be retained for historical or auditing purposes.
- **EXPIRED:** The entity has expired and is no longer valid. (e.g., a quota that has reached its expiration date)

ACTIVE = *'active'*

ARCHIVED = *'archived'*

BLOCKED = *'blocked'*

EXPIRED = *'expired'*

4.2.1 *class* UsageLog(*args: Any, **kwargs: Any)

Bases: **Base**

ORM model representing a single gateway usage record.

4.2.2 - **id**

- **Type:** The unique identifier for the usage log entry (primary key).

4.2.3 - **key_id**

- **Type:** The foreign key to the API key used for the request.

4.2.4 - **project_id**

- **Type:** The foreign key to the project billed for the request.

4.2.5 - **user_id**

- **Type:** The foreign key to the user associated with the request.

4.2.6 - **request_id**

- **Type:** Optional unique upstream or gateway request identifier.

4.2.7 - **timestamp**

- **Type:** The timestamp when the request was recorded.

- 4.2.8 - request_type**
 - **Type:** Optional request category.
- 4.2.9 - estimated_tokens**
 - **Type:** Optional token estimate captured before completion.
- 4.2.10 - prompt_tokens**
 - **Type:** Optional prompt token count.
- 4.2.11 - completion_tokens**
 - **Type:** Optional completion token count.
- 4.2.12 - total_tokens**
 - **Type:** Optional total and finalised token count.
- 4.2.13 - internal_cost_estimate**
 - **Type:** Optional pre-finalized internal cost estimate.
- 4.2.14 - internal_cost_final**
 - **Type:** Optional finalised internal cost.
- 4.2.15 - model**
 - **Type:** Optional model name used for the request.
- 4.2.16 - temperature**
 - **Type:** Optional sampling temperature parameter.
- 4.2.17 - top_p**
 - **Type:** Optional nucleus sampling parameter.
- 4.2.18 - top_k**
 - **Type:** Optional top-k sampling parameter.
- 4.2.19 - finish_reason**
 - **Type:** Optional completion stop reason.

4.2.20 - upstream_latency

- **Type:** Optional measured upstream service latency.

4.2.21 - gateway_overhead

- **Type:** Optional measured gateway processing overhead.

4.2.22 - total_latency

- **Type:** Optional total end-to-end request latency.

4.2.23 - ttft

- **Type:** Optional time-to-first-token for streaming responses.

4.2.24 - is_streaming

- **Type:** Optional flag indicating if response was streamed.

4.2.25 - status_code

- **Type:** Optional resulting HTTP status code.

Relationships

4.2.26 ————

4.2.27 - api_key

- **Type:** The relationship to the APIKey associated with this request.

4.2.28 - project

- **Type:** The relationship to the Project associated with this request.

4.2.29 - user

- **Type:** The relationship to the User associated with this request.

api_key = 'APIKey'

project = 'Project'

user = 'User'

4.2.30 *class* User(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a platform user.

4.2.31 - id

- **Type:** The unique identifier for the user (primary key).

4.2.32 - email

- **Type:** The unique email address used by the user.

4.2.33 - username

- **Type:** The unique public username.

4.2.34 - profile_picture_url

- **Type:** Optional URL to the user's profile picture.

4.2.35 - password_hash

- **Type:** The secure hash of the user's password.

4.2.36 - joined_date

- **Type:** The timestamp when the user account was created.

4.2.37 - last_login

- **Type:** Optional timestamp of the user's most recent login.

4.2.38 - password_expires_at

- **Type:** Optional timestamp for password expiration policy.

Relationships

4.2.39 ———-

4.2.40 - permissions

- **Type:** The relationship to ProjectPermission rows for this user.

4.2.41 - api_keys

- **Type:** The relationship to APIKey rows owned by this user.

4.2.42 - quotas

- **Type:** The relationship to Quota rows targeting this user.

`api_keys : sqlalchemy.orm.Mapped.list['APIKey'] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

4.2.43 hash_key(api_key: str, salt: str = None) → str

Hashes the provided API key using PBKDF2 HMAC with SHA-256. It generates a random salt if one is not provided, and returns a string containing the salt and the hash separated by a period.

- **Parameters:**
 - `api_key` (-)
 - `salt` (-)
- **Returns:** - str
- **Return type:** A string containing the hexadecimal representation of the salt and the hash, separated by a period.

4.2.44 verify_key(key_attempt: str, hashed_key: str) → bool

Verifies a key attempt against a stored hashed key hashed using the hash_key function. It extracts the salt and original hash from the stored hashed key, re-hashes the key attempt using the same salt, and compares the new hash with the original hash using a secure comparison method.

- **Parameters:**
 - `key_attempt` (-)
 - `hashed_key` (-)
- **Returns:** - bool
- **Return type:** True if the key attempt is valid and matches the stored hash, False otherwise.

4.3 Subpackages

- shared.config package
 - Submodules
 - shared.config.config module
- shared.db package

- `get_session()`
 - `get_transactional_session()`
 - Submodules
 - `shared.db.db` module
 - * `get_session()`
 - * `get_transactional_session()`
- `shared.models` package
 - `APIKey`
 - * `APIKey.Relationships`
 - * `APIKey.project`
 - * `APIKey.quotas`
 - * `APIKey.user`
 - `Limit`
 - * `Limit.Relationships`
 - * `Limit.quotas`
 - `Period`
 - * `Period.DAY`
 - * `Period.HOUR`
 - * `Period.MINUTE`
 - * `Period.MONTH`
 - * `Period.WEEK`
 - `Project`
 - * `Project.Relationships`
 - * `Project.api_keys`
 - * `Project.permissions`
 - * `Project.quotas`
 - `ProjectPermission`
 - * `ProjectPermission.Relationships`
 - * `ProjectPermission.project`
 - * `ProjectPermission.role`
 - * `ProjectPermission.user`
 - `Quota`
 - * `Quota.Relationships`
 - * `Quota.api_key`
 - * `Quota.limit`
 - * `Quota.limit_value`
 - * `Quota.project`
 - * `Quota.user`
 - `Role`
 - * `Role.Relationships`
 - * `Role.permissions`
 - `SQLAlchemyBase`

- Status
 - * Status.ACTIVE
 - * Status.ARCHIVED
 - * Status.BLOCKED
 - * Status.EXPIRED
- UsageLog
 - * UsageLog.Relationships
 - * UsageLog.api_key
 - * UsageLog.project
 - * UsageLog.user
- User
 - * User.Relationships
 - * User.api_keys
 - * User.permissions
 - * User.quotas
- Subpackages
 - * shared.models.orm_models package
 - APIKey
 - Base
 - Limit
 - Period
 - Project
 - ProjectPermission
 - Quota
 - Role
 - Status
 - UsageLog
 - User
 - Submodules
 - shared.models.orm_models.api_key module
 - shared.models.orm_models.base module
 - shared.models.orm_models.limit module
 - shared.models.orm_models.period_enum module
 - shared.models.orm_models.project module
 - shared.models.orm_models.project_permission module
 - shared.models.orm_models.quota module
 - shared.models.orm_models.role module
 - shared.models.orm_models.status_enum module
 - shared.models.orm_models.usage_log module
 - shared.models.orm_models.user module
- shared.utils package
 - calculate_date_after_period()
 - find_project_root()

- `hash_key()`
- `hash_password()`
- `verify_key()`
- `verify_password()`
- Submodules
- `shared.utils.date_calc` module
 - * `calculate_date_after_period()`
- `shared.utils.hashing` module
 - * `hash_key()`
 - * `verify_key()`
- `shared.utils.password` module
 - * `hash_password()`
 - * `verify_password()`
- `shared.utils.pathing` module
 - * `find_project_root()`

5 shared.models package

5.0.1 *class* APIKey(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing an API key in the database.

5.0.2 - id

- **Type:** The unique identifier for the API key (primary key).

5.0.3 - project_id

- **Type:** The foreign key linking to the associated project.

5.0.4 - user_id

- **Type:** The foreign key linking to the associated user, nullable for keys not tied to a specific user.

5.0.5 - name

- **Type:** A human-readable name for the API key.

5.0.6 - fingerprint

- **Type:** A unique fingerprint derived from the API key for quick lookup.

5.0.7 - key_hash

- **Type:** A secure hash of the API key for verification purposes.

5.0.8 - create_date

- **Type:** The timestamp when the API key was created.

5.0.9 - status

- **Type:** The current status of the API key (e.g., active, inactive).

Relationships

5.0.10 ————

5.0.11 - project

- **Type:** The relationship to the Project model, indicating which project this API key belongs to.

5.0.12 - user

- **Type:** The relationship to the User model, indicating which user this API key belongs to (if applicable).

5.0.13 - quotas

- **Type:** The relationship to the Quota model, representing the quotas directly targeting the API key.

`project = 'Project'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

`user = 'User'`

5.0.14 class Limit(*args: Any, **kwargs: Any)

Bases: `Base`

ORM model representing a quota limit type's definition.

5.0.15 - id

- **Type:** The unique identifier for the limit (primary key).

5.0.16 - name

- **Type:** A unique name for the limit type (for example ‘Token Limit’ or ‘Request Limit’).

5.0.17 - description

- **Type:** A human-readable description of what the limit controls.

Relationships

5.0.18 ————

5.0.19 - quotas

- **Type:** The relationship to Quota rows that reference this limit type.

quotas : *sqlalchemy.orm.Mapped.list*['Quota'] = 'Quota'

5.0.20 class Period(*values)

Bases: Enum

Enumeration of supported quota reset periods.

5.1 Values

- MINUTE
- HOUR
- DAY
- WEEK
- MONTH

DAY = 'day'

HOUR = 'hour'

MINUTE = 'minute'

MONTH = 'month'

WEEK = 'week'

5.1.1 class Project(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a project in the database.

5.1.2 - id

- **Type:** The unique identifier for the project (primary key).

5.1.3 - name

- **Type:** The unique, human-readable project name.

5.1.4 - status

- **Type:** The lifecycle status of the project (e.g., ACTIVE, ARCHIVED).

5.1.5 - created_date

- **Type:** The timestamp when the project was created.

5.1.6 - modified_date

- **Type:** The timestamp of the latest update, if any.

Relationships

5.1.7 _____

5.1.8 - permissions

- **Type:** The relationship to ProjectPermission rows linked to this project.

5.1.9 - api_keys

- **Type:** The relationship to APIKey rows issued under this project.

5.1.10 - quotas

- **Type:** The relationship to Quota rows directly targeting this project.

`api_keys : sqlalchemy.orm.Mapped[list['APIKey']] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped[list['ProjectPermission']] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped[list['Quota']] = 'Quota'`

5.1.11 `class ProjectPermission(*args: Any, **kwargs: Any)`

Bases: Base

ORM model representing a user's role assignment within a project.

5.1.12 - project_id

- **Type:** The foreign key to the associated project (composite primary key).

5.1.13 - user_id

- **Type:** The foreign key to the associated user (composite primary key).

5.1.14 - role_id

- **Type:** The foreign key to the role granted in the project.

5.1.15 - join_date

- **Type:** The timestamp when the permission assignment was created.

Relationships

5.1.16 ———-

5.1.17 - project

- **Type:** The relationship to the Project receiving the permission mapping.

5.1.18 - user

- **Type:** The relationship to the User receiving the role in the project.

5.1.19 - role

- **Type:** The relationship to the Role granted for the project membership.

project = *'Project'*

role = *'Role'*

user = *'User'*

5.1.20 *class* Quota(*args: Any, **kwargs: Any)

Bases: **Base**

ORM model representing quota allocation and reset state.

5.1.21 - id

- **Type:** The unique identifier for the quota entry (primary key).

5.1.22 - project_id

- **Type:** Optional foreign key to the project targeted by this quota.

5.1.23 - user_id

- **Type:** Optional foreign key to the user targeted by this quota.

5.1.24 - key_id

- **Type:** Optional foreign key to the API key targeted by this quota.

5.1.25 - limit_id

- **Type:** The foreign key to the limit type being constrained.

5.1.26 - limit_value

- **Type:** The allocated maximum value for the configured limit and period.

5.1.27 - period

- **Type:** The reset period for the quota.

5.1.28 - expires_at

- **Type:** Optional timestamp when this quota definition expires.

5.1.29 - status

- **Type:** The current lifecycle state of the quota.

5.1.30 - allocated

- **Type:** The currently consumed amount within the active period.

5.1.31 - next_reset

- **Type:** The timestamp when allocated usage resets.

Relationships

5.1.32 ————

5.1.33 - project

- **Type:** The relationship to the Project this quota directly applies to.

5.1.34 - `api_key`

- **Type:** The relationship to the `APIKey` this quota directly applies to.

5.1.35 - `user`

- **Type:** The relationship to the `User` this quota directly applies to.

5.1.36 - `limit`

- **Type:** The relationship to the `Limit` definition for this quota.

`api_key = 'APIKey'`

`limit = 'Limit'`

`limit_value : sqlalchemy.orm.Mapped.float / None`

`project = 'Project'`

`user = 'User'`

5.1.37 `class Role(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a project membership role's definition.

5.1.38 - `id`

- **Type:** The unique identifier for the role (primary key).

5.1.39 - `name`

- **Type:** A unique role name.

5.1.40 - `description`

- **Type:** A human-readable description of role permissions.

Relationships

5.1.41 ————

5.1.42 - `permissions`

- **Type:** The relationship to `ProjectPermission` rows using this role.

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

5.1.43 SQLAlchemyBase

alias of Base

5.1.44 class Status(*values)

Bases: Enum

Enumeration of lifecycle states for ORM entities.

5.2 Values

- **ACTIVE:** The entity is active and operational.
- **BLOCKED:** The entity is blocked and should not be used. (e.g., due to policy violations or security concerns)
- **ARCHIVED:** The entity is archived and no longer active. It may be retained for historical or auditing purposes.
- **EXPIRED:** The entity has expired and is no longer valid. (e.g., a quota that has reached its expiration date)

ACTIVE = *'active'*

ARCHIVED = *'archived'*

BLOCKED = *'blocked'*

EXPIRED = *'expired'*

5.2.1 class UsageLog(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a single gateway usage record.

5.2.2 - id

- **Type:** The unique identifier for the usage log entry (primary key).

5.2.3 - key_id

- **Type:** The foreign key to the API key used for the request.

5.2.4 - project_id

- **Type:** The foreign key to the project billed for the request.

5.2.5 - user_id

- **Type:** The foreign key to the user associated with the request.

5.2.6 - request_id

- **Type:** Optional unique upstream or gateway request identifier.

5.2.7 - timestamp

- **Type:** The timestamp when the request was recorded.

5.2.8 - request_type

- **Type:** Optional request category.

5.2.9 - estimated_tokens

- **Type:** Optional token estimate captured before completion.

5.2.10 - prompt_tokens

- **Type:** Optional prompt token count.

5.2.11 - completion_tokens

- **Type:** Optional completion token count.

5.2.12 - total_tokens

- **Type:** Optional total and finalised token count.

5.2.13 - internal_cost_estimate

- **Type:** Optional pre-finalized internal cost estimate.

5.2.14 - internal_cost_final

- **Type:** Optional finalised internal cost.

5.2.15 - model

- **Type:** Optional model name used for the request.

5.2.16 - temperature

- **Type:** Optional sampling temperature parameter.

5.2.17 - top_p

- **Type:** Optional nucleus sampling parameter.

5.2.18 - top_k

- **Type:** Optional top-k sampling parameter.

5.2.19 - finish_reason

- **Type:** Optional completion stop reason.

5.2.20 - upstream_latency

- **Type:** Optional measured upstream service latency.

5.2.21 - gateway_overhead

- **Type:** Optional measured gateway processing overhead.

5.2.22 - total_latency

- **Type:** Optional total end-to-end request latency.

5.2.23 - ttft

- **Type:** Optional time-to-first-token for streaming responses.

5.2.24 - is_streaming

- **Type:** Optional flag indicating if response was streamed.

5.2.25 - status_code

- **Type:** Optional resulting HTTP status code.

Relationships

5.2.26 ————

5.2.27 - api_key

- **Type:** The relationship to the APIKey associated with this request.

5.2.28 - project

- **Type:** The relationship to the Project associated with this request.

5.2.29 - user

- **Type:** The relationship to the User associated with this request.

`api_key = 'APIKey'`

`project = 'Project'`

`user = 'User'`

5.2.30 `class User(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a platform user.

5.2.31 - id

- **Type:** The unique identifier for the user (primary key).

5.2.32 - email

- **Type:** The unique email address used by the user.

5.2.33 - username

- **Type:** The unique public username.

5.2.34 - profile_picture_url

- **Type:** Optional URL to the user's profile picture.

5.2.35 - password_hash

- **Type:** The secure hash of the user's password.

5.2.36 - joined_date

- **Type:** The timestamp when the user account was created.

5.2.37 - last_login

- **Type:** Optional timestamp of the user's most recent login.

5.2.38 - password_expires_at

- **Type:** Optional timestamp for password expiration policy.

Relationships

5.2.39 —————

5.2.40 - permissions

- **Type:** The relationship to ProjectPermission rows for this user.

5.2.41 - api_keys

- **Type:** The relationship to APIKey rows owned by this user.

5.2.42 - quotas

- **Type:** The relationship to Quota rows targeting this user.

`api_keys : sqlalchemy.orm.Mapped[list['APIKey']] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped[list['ProjectPermission']] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped[list['Quota']] = 'Quota'`

5.3 Subpackages

- `shared.models.orm_models` package
 - `APIKey`
 - * `APIKey.Relationships`
 - * `APIKey.project`
 - * `APIKey.quotas`
 - * `APIKey.user`
 - `Base`
 - `Limit`
 - * `Limit.Relationships`
 - * `Limit.quotas`
 - `Period`
 - * `Period.DAY`
 - * `Period.HOUR`
 - * `Period.MINUTE`
 - * `Period.MONTH`
 - * `Period.WEEK`
 - `Project`
 - * `Project.Relationships`
 - * `Project.api_keys`
 - * `Project.permissions`
 - * `Project.quotas`
 - `ProjectPermission`

- * ProjectPermission.Relationships
 - * ProjectPermission.project
 - * ProjectPermission.role
 - * ProjectPermission.user
- Quota
 - * Quota.Relationships
 - * Quota.api_key
 - * Quota.limit
 - * Quota.limit_value
 - * Quota.project
 - * Quota.user
- Role
 - * Role.Relationships
 - * Role.permissions
- Status
 - * Status.ACTIVE
 - * Status.ARCHIVED
 - * Status.BLOCKED
 - * Status.EXPIRED
- UsageLog
 - * UsageLog.Relationships
 - * UsageLog.api_key
 - * UsageLog.project
 - * UsageLog.user
- User
 - * User.Relationships
 - * User.api_keys
 - * User.permissions
 - * User.quotas
- Submodules
- shared.models.orm_models.api_key module
 - * APIKey
 - APIKey.Relationships
 - APIKey.project
 - APIKey.quotas
 - APIKey.user
- shared.models.orm_models.base module
 - * Base
- shared.models.orm_models.limit module
 - * Limit
 - Limit.Relationships
 - Limit.quotas

- shared.models.orm_models.period_enum module
 - * Period
 - Period.DAY
 - Period.HOUR
 - Period.MINUTE
 - Period.MONTH
 - Period.WEEK
- shared.models.orm_models.project module
 - * Project
 - Project.Relationships
 - Project.api_keys
 - Project.permissions
 - Project.quotas
- shared.models.orm_models.project_permission module
 - * ProjectPermission
 - ProjectPermission.Relationships
 - ProjectPermission.project
 - ProjectPermission.role
 - ProjectPermission.user
- shared.models.orm_models.quota module
 - * Quota
 - Quota.Relationships
 - Quota.api_key
 - Quota.limit
 - Quota.limit_value
 - Quota.project
 - Quota.user
- shared.models.orm_models.role module
 - * Role
 - Role.Relationships
 - Role.permissions
- shared.models.orm_models.status_enum module
 - * Status
 - Status.ACTIVE
 - Status.ARCHIVED
 - Status.BLOCKED
 - Status.EXPIRED
- shared.models.orm_models.usage_log module
 - * UsageLog
 - UsageLog.Relationships
 - UsageLog.api_key

- UsageLog.project
- UsageLog.user
- shared.models.orm_models.user module
 - * User
 - User.Relationships
 - User.api_keys
 - User.permissions
 - User.quotas

6 shared.models.orm_models package

6.0.1 *class* APIKey(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing an API key in the database.

6.0.2 - id

- **Type:** The unique identifier for the API key (primary key).

6.0.3 - project_id

- **Type:** The foreign key linking to the associated project.

6.0.4 - user_id

- **Type:** The foreign key linking to the associated user, nullable for keys not tied to a specific user.

6.0.5 - name

- **Type:** A human-readable name for the API key.

6.0.6 - fingerprint

- **Type:** A unique fingerprint derived from the API key for quick lookup.

6.0.7 - key_hash

- **Type:** A secure hash of the API key for verification purposes.

6.0.8 - create_date

- **Type:** The timestamp when the API key was created.

6.0.9 - status

- **Type:** The current status of the API key (e.g., active, inactive).

Relationships

6.0.10 ————

6.0.11 - project

- **Type:** The relationship to the Project model, indicating which project this API key belongs to.

6.0.12 - user

- **Type:** The relationship to the User model, indicating which user this API key belongs to (if applicable).

6.0.13 - quotas

- **Type:** The relationship to the Quota model, representing the quotas directly targeting the API key.

`project = 'Project'`

`quotas : sqlalchemy.orm.Mapped[list['Quota']] = 'Quota'`

`user = 'User'`

6.0.14 `class Base(*args: Any, **kwargs: Any)`

Bases: `DeclarativeBase`

Base declarative class for all shared SQLAlchemy ORM models.

This class centralizes SQLAlchemy metadata registration so all ORM models under the shared package can be created and managed through a single base.

6.0.15 `class Limit(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a quota limit type's definition.

6.0.16 - id

- **Type:** The unique identifier for the limit (primary key).

6.0.17 - name

- **Type:** A unique name for the limit type (for example 'Token Limit' or 'Request Limit').

6.0.18 - description

- **Type:** A human-readable description of what the limit controls.

Relationships

6.0.19 ————

6.0.20 - quotas

- **Type:** The relationship to Quota rows that reference this limit type.

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

6.0.21 *class* Period(*values)

Bases: `Enum`

Enumeration of supported quota reset periods.

6.1 Values

- MINUTE
- HOUR
- DAY
- WEEK
- MONTH

`DAY = 'day'`

`HOUR = 'hour'`

`MINUTE = 'minute'`

`MONTH = 'month'`

`WEEK = 'week'`

6.1.1 *class* Project(*args: Any, **kwargs: Any)

Bases: `Base`

ORM model representing a project in the database.

6.1.2 - id

- **Type:** The unique identifier for the project (primary key).

6.1.3 - name

- **Type:** The unique, human-readable project name.

6.1.4 - status

- **Type:** The lifecycle status of the project (e.g., ACTIVE, ARCHIVED).

6.1.5 - created_date

- **Type:** The timestamp when the project was created.

6.1.6 - modified_date

- **Type:** The timestamp of the latest update, if any.

Relationships

6.1.7 ———-

6.1.8 - permissions

- **Type:** The relationship to ProjectPermission rows linked to this project.

6.1.9 - api_keys

- **Type:** The relationship to APIKey rows issued under this project.

6.1.10 - quotas

- **Type:** The relationship to Quota rows directly targeting this project.

`api_keys : sqlalchemy.orm.Mapped.list['APIKey'] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

6.1.11 class ProjectPermission(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a user's role assignment within a project.

6.1.12 - project_id

- **Type:** The foreign key to the associated project (composite primary key).

6.1.13 - user_id

- **Type:** The foreign key to the associated user (composite primary key).

6.1.14 - role_id

- **Type:** The foreign key to the role granted in the project.

6.1.15 - join_date

- **Type:** The timestamp when the permission assignment was created.

Relationships

6.1.16 ————

6.1.17 - project

- **Type:** The relationship to the Project receiving the permission mapping.

6.1.18 - user

- **Type:** The relationship to the User receiving the role in the project.

6.1.19 - role

- **Type:** The relationship to the Role granted for the project membership.

project = *'Project'*

role = *'Role'*

user = *'User'*

6.1.20 class Quota(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing quota allocation and reset state.

6.1.21 - id

- **Type:** The unique identifier for the quota entry (primary key).

6.1.22 - project_id

- **Type:** Optional foreign key to the project targeted by this quota.

6.1.23 - user_id

- **Type:** Optional foreign key to the user targeted by this quota.

6.1.24 - key_id

- **Type:** Optional foreign key to the API key targeted by this quota.

6.1.25 - limit_id

- **Type:** The foreign key to the limit type being constrained.

6.1.26 - limit_value

- **Type:** The allocated maximum value for the configured limit and period.

6.1.27 - period

- **Type:** The reset period for the quota.

6.1.28 - expires_at

- **Type:** Optional timestamp when this quota definition expires.

6.1.29 - status

- **Type:** The current lifecycle state of the quota.

6.1.30 - allocated

- **Type:** The currently consumed amount within the active period.

6.1.31 - next_reset

- **Type:** The timestamp when allocated usage resets.

Relationships

6.1.32 - _____

6.1.33 - project

- **Type:** The relationship to the Project this quota directly applies to.

6.1.34 - api_key

- **Type:** The relationship to the APIKey this quota directly applies to.

6.1.35 - user

- **Type:** The relationship to the User this quota directly applies to.

6.1.36 - limit

- **Type:** The relationship to the Limit definition for this quota.

`api_key = 'APIKey'`

`limit = 'Limit'`

`limit_value : sqlalchemy.orm.Mapped.float / None`

`project = 'Project'`

`user = 'User'`

6.1.37 *class* Role(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a project membership role's definition.

6.1.38 - id

- **Type:** The unique identifier for the role (primary key).

6.1.39 - name

- **Type:** A unique role name.

6.1.40 - description

- **Type:** A human-readable description of role permissions.

Relationships

6.1.41 ————

6.1.42 - permissions

- **Type:** The relationship to ProjectPermission rows using this role.

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

6.1.43 *class* Status(*values)

Bases: Enum

Enumeration of lifecycle states for ORM entities.

6.2 Values

- **ACTIVE:** The entity is active and operational.
- **BLOCKED:** The entity is blocked and should not be used. (e.g., due to policy violations or security concerns)
- **ARCHIVED:** The entity is archived and no longer active. It may be retained for historical or auditing purposes.
- **EXPIRED:** The entity has expired and is no longer valid. (e.g., a quota that has reached its expiration date)

ACTIVE = *'active'*

ARCHIVED = *'archived'*

BLOCKED = *'blocked'*

EXPIRED = *'expired'*

6.2.1 *class* UsageLog(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a single gateway usage record.

6.2.2 - id

- **Type:** The unique identifier for the usage log entry (primary key).

6.2.3 - key_id

- **Type:** The foreign key to the API key used for the request.

6.2.4 - project_id

- **Type:** The foreign key to the project billed for the request.

6.2.5 - user_id

- **Type:** The foreign key to the user associated with the request.

6.2.6 - request_id

- **Type:** Optional unique upstream or gateway request identifier.

6.2.7 - timestamp

- **Type:** The timestamp when the request was recorded.

6.2.8 - request_type

- **Type:** Optional request category.

6.2.9 - estimated_tokens

- **Type:** Optional token estimate captured before completion.

6.2.10 - prompt_tokens

- **Type:** Optional prompt token count.

6.2.11 - completion_tokens

- **Type:** Optional completion token count.

6.2.12 - total_tokens

- **Type:** Optional total and finalised token count.

6.2.13 - internal_cost_estimate

- **Type:** Optional pre-finalized internal cost estimate.

6.2.14 - internal_cost_final

- **Type:** Optional finalised internal cost.

6.2.15 - model

- **Type:** Optional model name used for the request.

6.2.16 - temperature

- **Type:** Optional sampling temperature parameter.

6.2.17 - top_p

- **Type:** Optional nucleus sampling parameter.

6.2.18 - top_k

- **Type:** Optional top-k sampling parameter.

6.2.19 - finish_reason

- **Type:** Optional completion stop reason.

6.2.20 - upstream_latency

- **Type:** Optional measured upstream service latency.

6.2.21 - gateway_overhead

- **Type:** Optional measured gateway processing overhead.

6.2.22 - total_latency

- **Type:** Optional total end-to-end request latency.

6.2.23 - ttft

- **Type:** Optional time-to-first-token for streaming responses.

6.2.24 - is_streaming

- **Type:** Optional flag indicating if response was streamed.

6.2.25 - status_code

- **Type:** Optional resulting HTTP status code.

Relationships

6.2.26 ———-

6.2.27 - api_key

- **Type:** The relationship to the APIKey associated with this request.

6.2.28 - project

- **Type:** The relationship to the Project associated with this request.

6.2.29 - user

- **Type:** The relationship to the User associated with this request.

api_key = 'APIKey'

project = 'Project'

user = 'User'

6.2.30 *class* User(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a platform user.

6.2.31 - id

- **Type:** The unique identifier for the user (primary key).

6.2.32 - email

- **Type:** The unique email address used by the user.

6.2.33 - username

- **Type:** The unique public username.

6.2.34 - profile_picture_url

- **Type:** Optional URL to the user's profile picture.

6.2.35 - password_hash

- **Type:** The secure hash of the user's password.

6.2.36 - joined_date

- **Type:** The timestamp when the user account was created.

6.2.37 - last_login

- **Type:** Optional timestamp of the user's most recent login.

6.2.38 - password_expires_at

- **Type:** Optional timestamp for password expiration policy.

Relationships

6.2.39 ———

6.2.40 - permissions

- **Type:** The relationship to ProjectPermission rows for this user.

6.2.41 - api_keys

- **Type:** The relationship to APIKey rows owned by this user.

6.2.42 - quotas

- **Type:** The relationship to Quota rows targeting this user.

`api_keys : sqlalchemy.orm.Mapped.list['APIKey'] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

6.3 Submodules

6.4 shared.models.orm_models.api_key module

6.4.1 `class APIKey(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing an API key in the database.

6.4.2 - id

- **Type:** The unique identifier for the API key (primary key).

6.4.3 - project_id

- **Type:** The foreign key linking to the associated project.

6.4.4 - user_id

- **Type:** The foreign key linking to the associated user, nullable for keys not tied to a specific user.

6.4.5 - name

- **Type:** A human-readable name for the API key.

6.4.6 - fingerprint

- **Type:** A unique fingerprint derived from the API key for quick lookup.

6.4.7 - key_hash

- **Type:** A secure hash of the API key for verification purposes.

6.4.8 - create_date

- **Type:** The timestamp when the API key was created.

6.4.9 - status

- **Type:** The current status of the API key (e.g., active, inactive).

Relationships

6.4.10 ———-

6.4.11 - project

- **Type:** The relationship to the Project model, indicating which project this API key belongs to.

6.4.12 - user

- **Type:** The relationship to the User model, indicating which user this API key belongs to (if applicable).

6.4.13 - quotas

- **Type:** The relationship to the Quota model, representing the quotas directly targeting the API key.

`project = 'Project'`

`quotas : sqlalchemy.orm.Mapped[list['Quota']] = 'Quota'`

`user = 'User'`

6.5 shared.models.orm_models.base module

6.5.1 `class Base(*args: Any, **kwargs: Any)`

Bases: `DeclarativeBase`

Base declarative class for all shared SQLAlchemy ORM models.

This class centralizes SQLAlchemy metadata registration so all ORM models under the shared package can be created and managed through a single base.

6.6 shared.models.orm_models.limit module

6.6.1 `class Limit(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a quota limit type's definition.

6.6.2 - id

- **Type:** The unique identifier for the limit (primary key).

6.6.3 - name

- **Type:** A unique name for the limit type (for example ‘Token Limit’ or ‘Request Limit’).

6.6.4 - description

- **Type:** A human-readable description of what the limit controls.

Relationships

6.6.5 ————

6.6.6 - quotas

- **Type:** The relationship to Quota rows that reference this limit type.

`quotas : sqlalchemy.orm.Mapped[list[‘Quota’]] = ‘Quota’`

6.7 shared.models.orm_models.period_enum module

6.7.1 *class* Period(*values)

Bases: Enum

Enumeration of supported quota reset periods.

6.7.2 Values

- MINUTE
- HOUR
- DAY
- WEEK
- MONTH

`DAY = ‘day’`

`HOUR = ‘hour’`

`MINUTE = ‘minute’`

`MONTH = ‘month’`

`WEEK = ‘week’`

6.8 shared.models.orm_models.project module

6.8.1 `class Project(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a project in the database.

6.8.2 - `id`

- **Type:** The unique identifier for the project (primary key).

6.8.3 - `name`

- **Type:** The unique, human-readable project name.

6.8.4 - `status`

- **Type:** The lifecycle status of the project (e.g., `ACTIVE`, `ARCHIVED`).

6.8.5 - `created_date`

- **Type:** The timestamp when the project was created.

6.8.6 - `modified_date`

- **Type:** The timestamp of the latest update, if any.

Relationships

6.8.7 `_____`

6.8.8 - `permissions`

- **Type:** The relationship to `ProjectPermission` rows linked to this project.

6.8.9 - `api_keys`

- **Type:** The relationship to `APIKey` rows issued under this project.

6.8.10 - `quotas`

- **Type:** The relationship to `Quota` rows directly targeting this project.

`api_keys : sqlalchemy.orm.Mapped[list['APIKey']] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped[list['ProjectPermission']] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped[list['Quota']] = 'Quota'`

6.9 shared.models.orm_models.project_permission module

6.9.1 *class* ProjectPermission(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a user's role assignment within a project.

6.9.2 - project_id

- **Type:** The foreign key to the associated project (composite primary key).

6.9.3 - user_id

- **Type:** The foreign key to the associated user (composite primary key).

6.9.4 - role_id

- **Type:** The foreign key to the role granted in the project.

6.9.5 - join_date

- **Type:** The timestamp when the permission assignment was created.

Relationships

6.9.6 ———-

6.9.7 - project

- **Type:** The relationship to the Project receiving the permission mapping.

6.9.8 - user

- **Type:** The relationship to the User receiving the role in the project.

6.9.9 - role

- **Type:** The relationship to the Role granted for the project membership.

project = *'Project'*

role = *'Role'*

user = *'User'*

6.10 shared.models.orm_models.quota module

6.10.1 *class* Quota(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing quota allocation and reset state.

6.10.2 - id

- **Type:** The unique identifier for the quota entry (primary key).

6.10.3 - project_id

- **Type:** Optional foreign key to the project targeted by this quota.

6.10.4 - user_id

- **Type:** Optional foreign key to the user targeted by this quota.

6.10.5 - key_id

- **Type:** Optional foreign key to the API key targeted by this quota.

6.10.6 - limit_id

- **Type:** The foreign key to the limit type being constrained.

6.10.7 - limit_value

- **Type:** The allocated maximum value for the configured limit and period.

6.10.8 - period

- **Type:** The reset period for the quota.

6.10.9 - expires_at

- **Type:** Optional timestamp when this quota definition expires.

6.10.10 - status

- **Type:** The current lifecycle state of the quota.

6.10.11 - allocated

- **Type:** The currently consumed amount within the active period.

6.10.12 - next_reset

- **Type:** The timestamp when allocated usage resets.

Relationships

6.10.13 `_____`

6.10.14 - `project`

- **Type:** The relationship to the Project this quota directly applies to.

6.10.15 - `api_key`

- **Type:** The relationship to the APIKey this quota directly applies to.

6.10.16 - `user`

- **Type:** The relationship to the User this quota directly applies to.

6.10.17 - `limit`

- **Type:** The relationship to the Limit definition for this quota.

`api_key = 'APIKey'`

`limit = 'Limit'`

`limit_value : sqlalchemy.orm.Mapped.float / None`

`project = 'Project'`

`user = 'User'`

6.11 `shared.models.orm_models.role` module

6.11.1 `class Role(*args: Any, **kwargs: Any)`

Bases: `Base`

ORM model representing a project membership role's definition.

6.11.2 - `id`

- **Type:** The unique identifier for the role (primary key).

6.11.3 - `name`

- **Type:** A unique role name.

6.11.4 - `description`

- **Type:** A human-readable description of role permissions.

Relationships

6.11.5 ————

6.11.6 - permissions

- **Type:** The relationship to ProjectPermission rows using this role.

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

6.12 shared.models.orm_models.status_enum module

6.12.1 class Status(*values)

Bases: Enum

Enumeration of lifecycle states for ORM entities.

6.12.2 Values

- **ACTIVE:** The entity is active and operational.
- **BLOCKED:** The entity is blocked and should not be used. (e.g., due to policy violations or security concerns)
- **ARCHIVED:** The entity is archived and no longer active. It may be retained for historical or auditing purposes.
- **EXPIRED:** The entity has expired and is no longer valid. (e.g., a quota that has reached its expiration date)

ACTIVE = *'active'*

ARCHIVED = *'archived'*

BLOCKED = *'blocked'*

EXPIRED = *'expired'*

6.13 shared.models.orm_models.usage_log module

6.13.1 class UsageLog(*args: Any, **kwargs: Any)

Bases: Base

ORM model representing a single gateway usage record.

6.13.2 - id

- **Type:** The unique identifier for the usage log entry (primary key).

6.13.3 - key_id

- **Type:** The foreign key to the API key used for the request.

6.13.4 - project_id

- **Type:** The foreign key to the project billed for the request.

6.13.5 - user_id

- **Type:** The foreign key to the user associated with the request.

6.13.6 - request_id

- **Type:** Optional unique upstream or gateway request identifier.

6.13.7 - timestamp

- **Type:** The timestamp when the request was recorded.

6.13.8 - request_type

- **Type:** Optional request category.

6.13.9 - estimated_tokens

- **Type:** Optional token estimate captured before completion.

6.13.10 - prompt_tokens

- **Type:** Optional prompt token count.

6.13.11 - completion_tokens

- **Type:** Optional completion token count.

6.13.12 - total_tokens

- **Type:** Optional total and finalised token count.

6.13.13 - internal_cost_estimate

- **Type:** Optional pre-finalized internal cost estimate.

6.13.14 - internal_cost_final

- **Type:** Optional finalised internal cost.

6.13.15 - model

- **Type:** Optional model name used for the request.

6.13.16 - temperature

- **Type:** Optional sampling temperature parameter.

6.13.17 - top_p

- **Type:** Optional nucleus sampling parameter.

6.13.18 - top_k

- **Type:** Optional top-k sampling parameter.

6.13.19 - finish_reason

- **Type:** Optional completion stop reason.

6.13.20 - upstream_latency

- **Type:** Optional measured upstream service latency.

6.13.21 - gateway_overhead

- **Type:** Optional measured gateway processing overhead.

6.13.22 - total_latency

- **Type:** Optional total end-to-end request latency.

6.13.23 - ttft

- **Type:** Optional time-to-first-token for streaming responses.

6.13.24 - is_streaming

- **Type:** Optional flag indicating if response was streamed.

6.13.25 - status_code

- **Type:** Optional resulting HTTP status code.

Relationships

6.13.26 **_____**

6.13.27 - **api_key**

- **Type:** The relationship to the APIKey associated with this request.

6.13.28 - **project**

- **Type:** The relationship to the Project associated with this request.

6.13.29 - **user**

- **Type:** The relationship to the User associated with this request.

api_key = *'APIKey'*

project = *'Project'*

user = *'User'*

6.14 shared.models.orm_models.user module

6.14.1 *class* **User**(*args: Any, **kwargs: Any)

Bases: **Base**

ORM model representing a platform user.

6.14.2 - **id**

- **Type:** The unique identifier for the user (primary key).

6.14.3 - **email**

- **Type:** The unique email address used by the user.

6.14.4 - **username**

- **Type:** The unique public username.

6.14.5 - **profile_picture_url**

- **Type:** Optional URL to the user's profile picture.

6.14.6 - **password_hash**

- **Type:** The secure hash of the user's password.

6.14.7 - joined_date

- **Type:** The timestamp when the user account was created.

6.14.8 - last_login

- **Type:** Optional timestamp of the user's most recent login.

6.14.9 - password_expires_at

- **Type:** Optional timestamp for password expiration policy.

Relationships

6.14.10 ————

6.14.11 - permissions

- **Type:** The relationship to ProjectPermission rows for this user.

6.14.12 - api_keys

- **Type:** The relationship to APIKey rows owned by this user.

6.14.13 - quotas

- **Type:** The relationship to Quota rows targeting this user.

`api_keys : sqlalchemy.orm.Mapped.list['APIKey'] = 'APIKey'`

`permissions : sqlalchemy.orm.Mapped.list['ProjectPermission'] = 'ProjectPermission'`

`quotas : sqlalchemy.orm.Mapped.list['Quota'] = 'Quota'`

7 shared.utils package

7.0.1 `calculate_date_after_period(period: Period, start_date: datetime | None = None, fast_forward: bool = False) → datetime`

Calculate the next date after a given period from the start date.

- **Parameters:**
 - `period` (-)
 - `start_date` (-)
 - `fast_forward` (-)

- **Return type:**

- The calculated date after the specified period.

7.0.2 `find_project_root(start: Path = None, marker: str = 'shared/') → Path`

Tries to find the project root directory by looking for a specific marker file or directory (default is “shared/”) starting from the given path and moving up the directory tree until it finds the marker or reaches the filesystem root. Used to locate the project root directory inside Docker containers where folder structures may differ.

- **Parameters:**

- `start` (-)
- `marker` (-)

- **Returns: - Path**

- **Return type:** The path to the project root directory.

7.0.3 `hash_key(api_key: str, salt: str = None) → str`

Hashes the provided API key using PBKDF2 HMAC with SHA-256. It generates a random salt if one is not provided, and returns a string containing the salt and the hash separated by a period.

- **Parameters:**

- `api_key` (-)
- `salt` (-)

- **Returns: - str**

- **Return type:** A string containing the hexadecimal representation of the salt and the hash, separated by a period.

7.0.4 `hash_password(password: str) → str`

Hashes a plaintext password using Argon2 and returns the resulting hash string. The returned hash includes all necessary parameters and salt for later verification.

- **Parameters:** `password` (-)

7.0.5 `verify_key(key_attempt: str, hashed_key: str) → bool`

Verifies a key attempt against a stored hashed key hashed using the `hash_key` function. It extracts the salt and original hash from the stored hashed key, re-hashes the key attempt using the same salt, and compares the new hash with the original hash using a secure comparison method.

- **Parameters:**
 - `key_attempt` (-)
 - `hashed_key` (-)
- **Returns:** - bool
- **Return type:** True if the key attempt is valid and matches the stored hash, False otherwise.

7.0.6 `verify_password(stored_hash: str, provided_password: str) → bool`

Verifies a provided password against a stored hash using Argon2.

- **Parameters:**
 - `stored_hash` (-)
 - `provided_password` (-)
- **Returns:** - bool
- **Return type:** True if the provided password is valid and matches the stored hash, False otherwise.

7.1 Submodules

7.2 `shared.utils.date_calc` module

7.2.1 `calculate_date_after_period(period: Period, start_date: datetime | None = None, fast_forward: bool = False) → datetime`

Calculate the next date after a given period from the start date.

- **Parameters:**
 - `period` (-)
 - `start_date` (-)
 - `fast_forward` (-)
- **Return type:**
 - The calculated date after the specified period.

7.3 `shared.utils.hashing` module

7.3.1 `hash_key(api_key: str, salt: str = None) → str`

Hashes the provided API key using PBKDF2 HMAC with SHA-256. It generates a random salt if one is not provided, and returns a string containing the salt and the hash separated by a period.

- **Parameters:**

- **api_key** (-)
- **salt** (-)
- **Returns:** - **str**
- **Return type:** A string containing the hexadecimal representation of the salt and the hash, separated by a period.

7.3.2 **verify_key(key_attempt: str, hashed_key: str) → bool**

Verifies a key attempt against a stored hashed key hashed using the `hash_key` function. It extracts the salt and original hash from the stored hashed key, re-hashes the key attempt using the same salt, and compares the new hash with the original hash using a secure comparison method.

- **Parameters:**
 - **key_attempt** (-)
 - **hashed_key** (-)
- **Returns:** - **bool**
- **Return type:** True if the key attempt is valid and matches the stored hash, False otherwise.

7.4 **shared.utils.password module**

7.4.1 **hash_password(password: str) → str**

Hashes a plaintext password using Argon2 and returns the resulting hash string. The returned hash includes all necessary parameters and salt for later verification.

- **Parameters:** **password** (-)

7.4.2 **verify_password(stored_hash: str, provided_password: str) → bool**

Verifies a provided password against a stored hash using Argon2.

- **Parameters:**
 - **stored_hash** (-)
 - **provided_password** (-)
- **Returns:** - **bool**
- **Return type:** True if the provided password is valid and matches the stored hash, False otherwise.

7.5 shared.utils.pathing module

7.5.1 `find_project_root(start: Path = None, marker: str = 'shared/') → Path`

Tries to find the project root directory by looking for a specific marker file or directory (default is “shared/”) starting from the given path and moving up the directory tree until it finds the marker or reaches the filesystem root. Used to locate the project root directory inside Docker containers where folder structures may differ.

- **Parameters:**

- **start** (-)
- **marker** (-)

- **Returns: - Path**

- **Return type:** The path to the project root directory.